

Lecture 1: Course Introduction

Networks, Data and Basic Terminology

Elvin Isufi

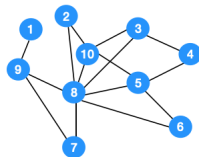
Intelligent Systems Department

September 3, 2025

Networks and graphs

- **Networks and graphs:** A mathematical representation of connected things

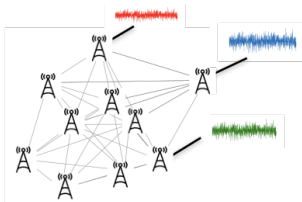
- ⇒ **Ex.** Connections of people in a social network
- ⇒ **Ex.** Interactions of complex systems
- ⇒ **Ex.** A water network, a power network, ...



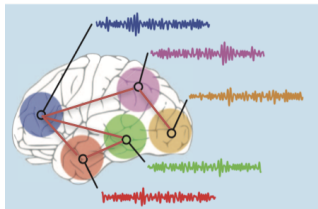
- Capture the **rich and complex** connectivity information
- Networks and graphs are **mathematical representations** of complex systems
 - ⇒ Understand **complex systems** $\Leftrightarrow$ Understand **networks** behind them
- Networks are **complex data**
 - ⇒ Processing **complex data** $\Leftrightarrow$ Capturing the **network structure** behind them

Why networks?

- They are observed everywhere
 - ⇒ Framework for representing complex data relationships
 - ⇒ Capture nontrivial/ higher-order relationships/dependencies between nodes



Communication: **Nodes:** base stations,
Edges: Communication links; **Task:**
allocate resources, detect anomalies



Brain: **Nodes:** brain areas, **Edges:**
Structural/functional connectivity;
Task: detect diseases (classify
patients), understand the behavior

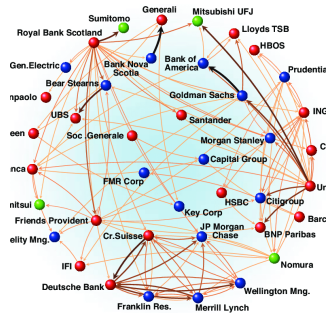


Social: **Nodes:** users, **Edges:**
friendships; **Task:** recommend friends,
detect fake accounts

Why networks? (cont...)



Water networks: **Nodes:** houses, junctions, **Edges:** pipes; **Task:** detect leakages, predict water demands



Financial: **Nodes:** companies, **Edges:** transactions; **Task:** detect fraud



Transport: **Nodes:** train stations, **Edges:** railway route; **Task:** predict delays

Economic impact

- ▶ Google Market cap: \$1.24 trillion
- ▶ Facebook Market cap: \$736 billion
- ▶ Cisco Market cap: \$188 billion
- ▶ Apple Market cap: \$2.22 trillion

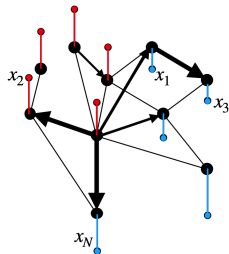


Image credit: J. Leskovec

Network data

► Scalar data over nodes:

- ⇒ **Nodes**: users of a recommender systems
- ⇒ **Edges**: similarities between them (e.g., Pearson correlation)
- ⇒ **Node data**: the rating, 1-5, given to a movie



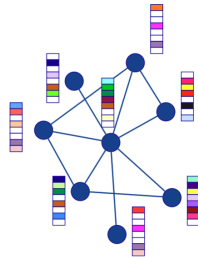
► Node data as a vector:

- ⇒ a.k.a. **graph signal**: $\mathbf{x} = [1, 0, 3, 2, 3, 4, 4, 5]^T \in \mathbb{R}^N$
- ⇒ entry x_i is the signal value associated to node i

Network data

► Vector data over nodes:

- ⇒ **Nodes**: users of a recommender systems
- ⇒ **Edges**: similarities between them (e.g., Pearson correlation)
- ⇒ **Node data features**: user features: name, age, location, profession....



► Node data features as a matrix: $\mathbf{X} = \begin{bmatrix} x_{11} & \dots & x_{1F} \\ x_{21} & \dots & x_{2F} \\ \dots & \dots & \dots \\ x_{N1} & \dots & x_{NF} \end{bmatrix} \in \mathbb{R}^{N \times F}$

⇒ **row i** are the features of node i : $\mathbf{x}_i = [x_{i1}, \dots, x_{iF}]^\top \in \mathbb{R}^F$

⇒ **column f** is the f th graph signal over all nodes: $\mathbf{x}_{:f} = [x_{1f}, \dots, x_{Nf}]^\top \in \mathbb{R}^N$

Network data

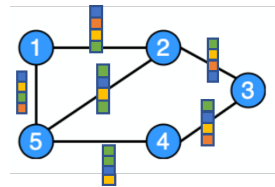
► Data over edges:

⇒ **Nodes**: junctions of a water networks

⇒ **Edges**: pipes connecting junctions

⇒ **Edge data features**: water flow at the pipe; type of pipe; diameter

...



► Edge data features as a matrix: $\mathbf{F} = \begin{bmatrix} f_{11} & \dots & f_{1F} \\ f_{21} & \dots & f_{2F} \\ \dots & \dots & \dots \\ f_{E1} & \dots & f_{EF} \end{bmatrix} \in \mathbb{R}^{E \times F}$

⇒ **row e** are the features of edge e : $\mathbf{f}_{e:} = [f_{e1}, \dots, f_{eF}]^\top \in \mathbb{R}^F$

⇒ **column g** collect the g th feature for all edges: $\mathbf{f}_{:g} = [f_{1g}, \dots, f_{Eg}]^\top \in \mathbb{R}^E$

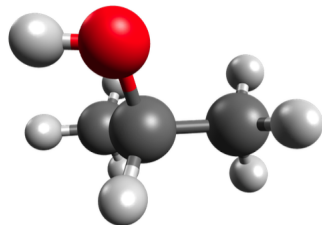
Network data

► Data data over the whole network:

⇒ **Nodes:** atoms

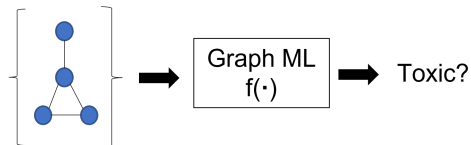
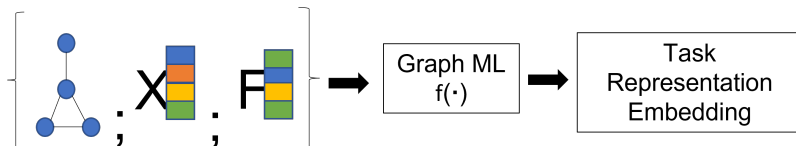
⇒ **Edges:** bonds between them

⇒ **Data over the network:** type of molecule; category of it; or a label



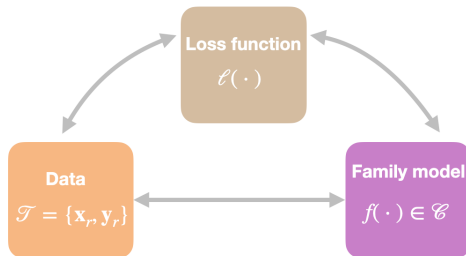
What is graph machine learning?

- **Goal:** Find the best function for the task we want to solve; i.e., find a mapping function $f(\cdot)$



What is graph machine learning?

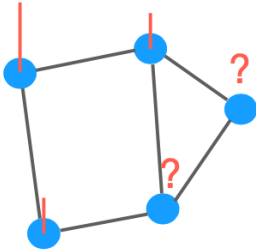
- **Goal:** Find the best function for the task we want to solve; i.e., optimize function $f(\cdot)$



- ⇒ **Need data:** graph structure **and possibly** node features, edge features, graph features
- ⇒ **Need function $f(\cdot)$** that leverages the graph structure
- ⇒ **Tackle the curse of dimensionality:** work with little data and large networks
- ⇒ **Need a loss function** for the task of interest solve

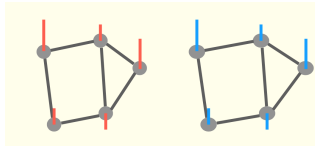
Landmark tasks with graph machine learning

Signal-level tasks



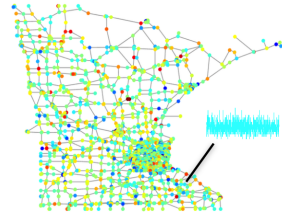
Signal reconstruction/interpolation

Ex. Find missing values in a sensor network



Signal classification

Ex. Health/non-healthy patient from EEG recordings



Signal prediction

Ex. Predict traffic

Landmark tasks with graph machine learning

Examples of Signal-level tasks

- Air pollution monitoring with Internet-of-Things

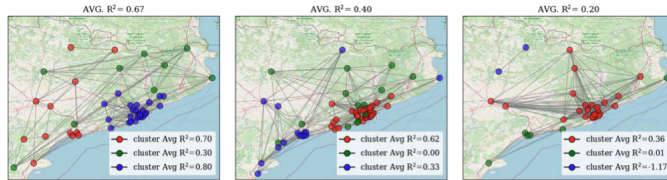


Image credit: Graph Signal Reconstruction Techniques for IoT Air Pollution Monitoring Platforms

- Authorship attribution from word adjacencies (course project for ChatGPT)

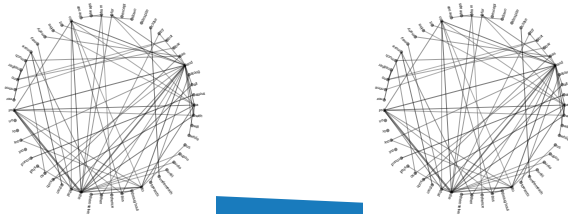
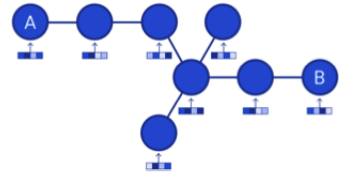
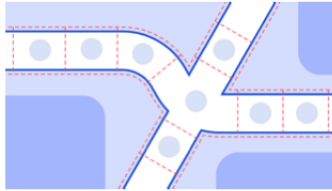
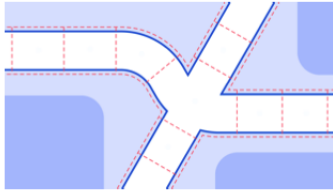


Image credit: Attributing the Authorship of the Henry VI Plays by Word Adjacency

Landmark tasks with graph machine learning

Examples of Signal-level tasks

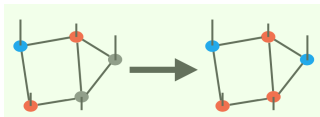


- Predict estimation time-of-arrival on GoogleMaps
 - ⇒ **Nodes:** road segments
 - ⇒ **Edges:** proximity between segments
 - ⇒ **Task:** forecast signal

Image credit Deepmind: <https://www.deepmind.com/publications/eta-prediction-with-graph-neural-networks-in-google-maps>

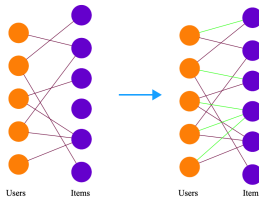
Landmark tasks with graph machine learning

Structure-level tasks



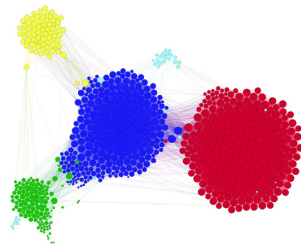
Node classification

Ex. Classify users of a social network from their friends labels



Link prediction

Ex. Recommend items



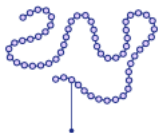
Clustering & community detection

Ex. Finding communities in facebook (same interest)

Landmark tasks with graph machine learning

Examples of Node-level tasks

Every protein is made up of a sequence of amino acids bonded together

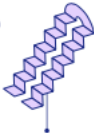


Amino acids

These amino acids interact locally to form shapes like helices and sheets



Alpha helix



Pleated sheet

These shapes fold up on larger scales to form the full three-dimensional protein structure



Pleated sheet

Alpha helix

Proteins can interact with other proteins, performing functions such as signalling and transcribing DNA



- Predict 3D protein structure based on its amino-acid sequence
 - ⇒ **Nodes:** amino-acids in a protein sequence
 - ⇒ **Edges:** proximity between amino-acids (residues)
 - ⇒ **Task:** predict node positions

Image credit: Deepmind: <https://www.deepmind.com/blog/alphafold-using-ai-for-scientific-discovery-2020>

Landmark tasks with graph machine learning

Examples of Node-level tasks

Median Free-Modelling Accuracy



AlphaFold's AI could change the world of biological science as we know it

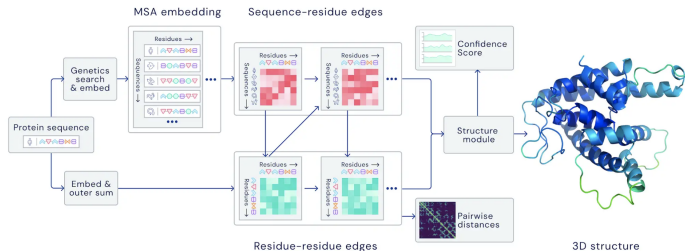
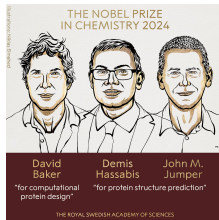
DeepMind's latest AI breakthrough can accurately predict the way proteins fold

Has Artificial Intelligence 'Solved' Biology's Protein-Folding Problem?

12-14-20

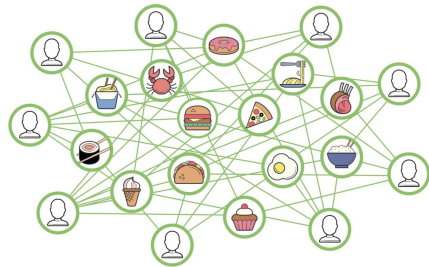
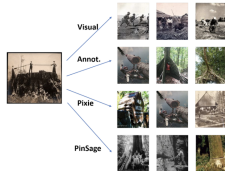
DeepMind's latest AI breakthrough could turbocharge drug discovery

Breakthrough in 2025



Landmark tasks with graph machine learning

Examples of Edge-level tasks



- Pin recommendation at **Pinterest** & food recommendation at **UberEats**

⇒ Billions of nodes, users, pins, restaurants, dishes - all in real time

⇒ <https://medium.com/pinterest-engineering/pinsage-a-new-graph-convolutional-neural-network-for-web-scale-recommender-systems-88795a107f48>

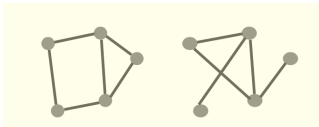
⇒ <https://www.uber.com/en-NL/blog/uber-eats-graph-learning/>

Graph ML powers LinkedIn



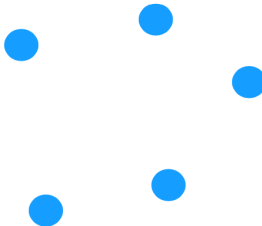
Landmark tasks with graph machine learning

Graph-level tasks



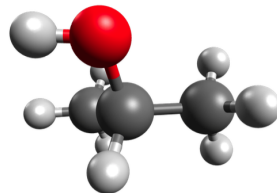
Graph classification

Ex. Given a collection of structures,
find which water network is resilient or
not



Graph identification

Ex. Given datapoints (stock features)
find how they influence each other

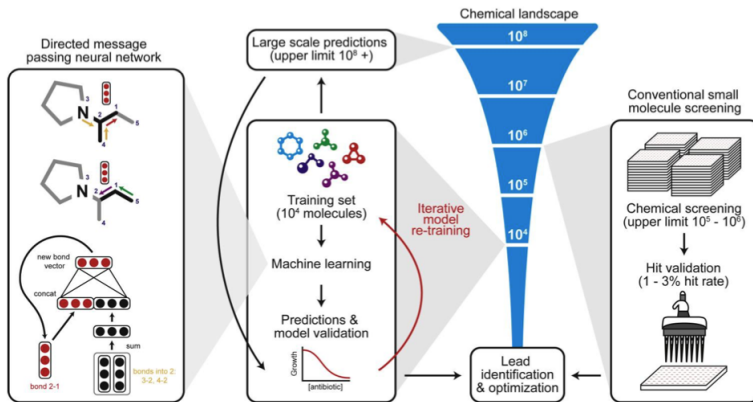


Graph generation

Ex. Given antibacterial activity level,
generate a new molecule

Landmark tasks with graph machine learning

Example graph-level tasks



- Analyse and process molecular structure to generate new antibiotics

⇒ https://www.youtube.com/watch?v=d7CkzgHi_tE&list=PLcZ0nmyqlala5P0skgoEkX0U729yFdYHF&index=5

Learning activity

Create your network data task

► Instructions

- ⇒ Groups of 2-3: Think of a **concrete real-world network data problem**
- ⇒ What is the high-level problem to solve?
- ⇒ What are the **nodes, edges, signals/features**?
- ⇒ How would you approach the problem from a network data task perspective?
- ⇒ What would be the main challenge of this approach?

► Logistics

- ⇒ **Time:** 10 min
- ⇒ **Goal:** Detail a problem from a network data perspective
- ⇒ **Product:** One example

Instructors



Dr. Elvin Isufi



Dr. Megha Khosla

Teaching Assistants



PhD: Tianqi Zhao



PhD: Mohammad Sabbaqi



PhD: Mohamed Jebali

- ▶ Teaching team - to students
 - ⇒ via Brightspace posts
- ▶ Students to Instructors
 - ⇒ During and after class (no office hours)
 - ⇒ **Limit emails to essentials** – reach out to us during class

- 12 Lectures - only in class - 3 Labs (support in class) - 1 final project

⇒ Dates and deadlines on BrightSpace

⇒ Submission via Brightspace

Fundamental Graph Machine Learning Component	Lec 1 - Course Introduction	Lab 1
	Lec 2 - Graph ML via Regularization	
	Lec 3 - Traditional Graph ML Methods	
	Lec 4 - Unsupervised Graph Representation Learning	
Graph Neural Networks	Lec 5 - Graph Convolutional Neural Networks	Lab 2
	Lec 6 - GNN Architectures	
	Lec 7 - Spectral Analysis	
	Lec 8 - Graph-Time Learning	
	Lec 9 - Learning over Temporal Graphs	
Challenges of Graph Machine Learning	Lec 10 - Scalability	Lab 3
	Lec 11 - Interpretability	
	Lec 12 - Privacy	
	Lec 13 - Guest Lecture	

- Guest lecture: TBD!

► Assessment

- ⇒ **Labs:** optional – solve them in team. Practice for the main project.
- ⇒ **Project:** groups 100% of the grade (report + individual oral exam)
- ⇒ Rubrics on Brightspace / assessment
- ⇒ **Start forming your project teams**

► Assessment

- ⇒ **Labs**: optional – solve them in team. Practice for the main project.
- ⇒ **Project**: groups 100% of the grade (report + individual oral exam)
- ⇒ Rubrics on Brightspace / assessment
- ⇒ **Start forming your project teams**

► Project meetings

- ⇒ A midterm meeting with the teaching team - **student team** schedules it with the teaching team

► Assessment

- ⇒ **Labs**: optional – solve them in team. Practice for the main project.
- ⇒ **Project**: groups 100% of the grade (report + individual oral exam)
- ⇒ Rubrics on Brightspace / assessment
- ⇒ **Start forming your project teams**

► Project meetings

- ⇒ A midterm meeting with the teaching team - **student team** schedules it with the teaching team

► Computing power

- ⇒ Labs and projects could be developed on a local PC - no need for huge power
- ⇒ We still recommend using DelftBlue for the project <https://www.tudelft.nl/dhpc/system>
- ⇒ Teaching staff does not give DelftBlue support
- ⇒ Contact official channels but documentation is detailed and easy to follow

The course targets CS master students with minimal dependencies on other courses
However, the following basic concepts are necessary

► Concepts

- ⇒ Basics of linear algebra: matrix multiplication, decompositions
- ⇒ Basics of machine learning: regression, classification, optimization, regularizer
- ⇒ Multilayer perceptron: what it is and how it works

The course targets CS master students with minimal dependencies on other courses
However, the following basic concepts are necessary

► Concepts

- ⇒ Basics of linear algebra: matrix multiplication, decompositions
- ⇒ Basics of machine learning: regression, classification, optimization, regularizer
- ⇒ Multilayer perceptron: what it is and how it works

► Software

- ⇒ Python: You need to have coding skills in Python
- ⇒ Some tutorials are provided on BrightSpace
- ⇒ Pytorch (or equivalent): You have coded once a simple deep learning model
- ⇒ We recommend Pytorch but other tools are fine
- ⇒ Tutorials on Brightspace

The course targets CS master students with minimal dependencies on other courses
However, the following basic concepts are necessary

► Concepts

- ⇒ Basics of linear algebra: matrix multiplication, decompositions
- ⇒ Basics of machine learning: regression, classification, optimization, regularizer
- ⇒ Multilayer perceptron: what it is and how it works

► Software

- ⇒ Python: You need to have coding skills in Python
- ⇒ Some tutorials are provided on BrightSpace
- ⇒ Pytorch (or equivalent): You have coded once a simple deep learning model
- ⇒ We recommend Pytorch but other tools are fine
- ⇒ Tutorials on Brightspace

Strongly recommended to complete the Python and Pytorch tutorials before Lab 1 if needed

Student feedback

- ▶ The assignments **could** be more supportive to achieve the course objectives
 - ⇒ **Action:** We provided theoretical sample questions!
 - ⇒ Prepare them and bring written notes to consult during the oral exam

Student feedback

- ▶ The assignments **could** be more supportive to achieve the course objectives
 - ⇒ **Action:** We provided theoretical sample questions!
 - ⇒ Prepare them and bring written notes to consult during the oral exam
- ▶ **Workload on labs**
 - ⇒ **Action:** Shortened assignments – moved some tasks to optional
 - ⇒ **Action:** Reduce computation time for some labs
 - ⇒ **Action:** Provide guidelines to prepare the environment

Student feedback

- ▶ The assignments **could** be more supportive to achieve the course objectives
 - ⇒ **Action:** We provided theoretical sample questions!
 - ⇒ Prepare them and bring written notes to consult during the oral exam
- ▶ Workload on labs
 - ⇒ **Action:** Shortened assignments – moved some tasks to optional
 - ⇒ **Action:** Reduce computation time for some labs
 - ⇒ **Action:** Provide guidelines to prepare the environment
- ▶ More project ideas
 - ⇒ **Action:** We provide now 5 projects you can pick from
 - ⇒ **We recommend you come up with your own project – part of the fun/learning**
 - ⇒ You can always consult us to refine the idea

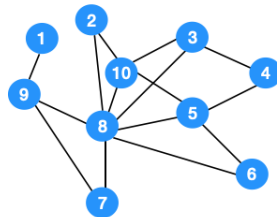
Outline: What will you learn today?

- ▶ How to frame a problem with the graph language?
 - ⇒ Graph terminology
 - ⇒ Graph matrices
 - ⇒ Features
 - ⇒ How to build graphs?

Graph terminology

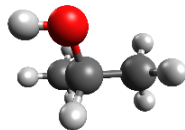
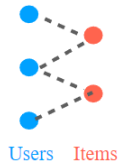
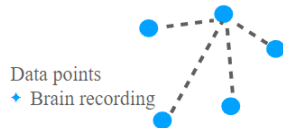
Graph Terminology

- ▶ Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}) \Rightarrow$ A set $\mathcal{V}$ of vertices or nodes
 $\Rightarrow$ Connected by a set $\mathcal{E} = \{(i, j)\}$ of edges or links
- ▶ In figure $\Rightarrow$ Vertices are $\mathcal{V} = \{1, 2, 3, \dots, 10\}$
 $\Rightarrow$ Edges $\mathcal{E} = \{(1, 9), (2, 8), (2, 10), (3, 4), \dots\}$
- ▶ We say graph $\mathcal{G}$ has order $N := |\mathcal{V}|$ nodes, and size $M := |\mathcal{E}|$ edges
- ▶ Neighbor of node i is $\mathcal{N}_i$
 $\Rightarrow \mathcal{N}_4 = \{3, 5\}$
 $\Rightarrow \mathcal{N}_8 = \{2, 3, 5, 6, 7, 9, 10\}$



Setting an edge

- ▶ **Pairwise similarity** between data points
⇒ Pearson correlation / cosine similarity
- ▶ **Interaction** between entities
⇒ User-item in a recommender system
- ▶ **Physical connectivity**
⇒ communication link, molecular bond



Vertices and edges in networks

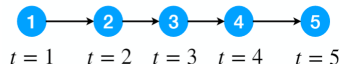
Network	Vertex	Edge
Internet	Computer/router	Cable or wireless link
Metabolic network	Metabolite	Metabolic reaction
WWW	Web page	Hyperlink
Food web	Species	Predation
Gene-regulatory network	Gene	Regulation of expression
Friendship network	Person	Friendship or acquaintance
Power grid	Substation	Transmission line
Affiliation network	Person and club	Membership
Protein interaction	Protein	Physical interaction
Citation network	Article/patent	Citation
Neural network	Neuron	Synapse
⋮	⋮	⋮

Image credit: G. Mateos

Time and space as graphs

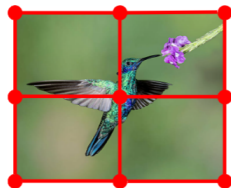
- Time as a graph

- ⇒ Nodes are time instants
- ⇒ Edges are temporal adjacencies



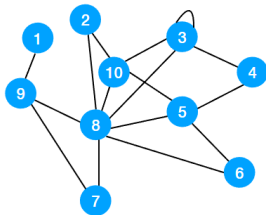
- Image as a grid graph

- ⇒ Nodes are pixels
- ⇒ Edges are spatial proximities between pixels

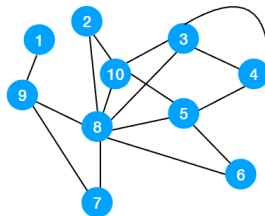


Simple and multi-graphs

- ▶ Graphs may have **self-loops** and **multi-edges**
 - ⇒ These are called multi-graphs
- ▶ We will work with simple graphs, with no self-loops or multi-edges



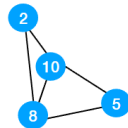
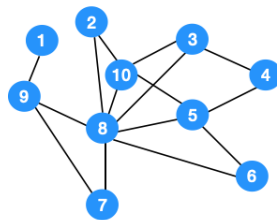
self-loops



multiple edges

Subgraphs

- ▶ Consider a given graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
- ▶ **Def:** Graph $\mathcal{G}' = (\mathcal{V}', \mathcal{E}')$ is an **induced subgraph** of $\mathcal{G}$ if:
 - ▶ $\mathcal{V}' \subseteq \mathcal{V}$
 - ▶ $\mathcal{E}' \subseteq \mathcal{E}$ is the collection of edges in $\mathcal{G}$ among that subset of vertices
- ▶ **Ex:** Graph induced by $\mathcal{V}' = \{2, 5, 8, 10\}$

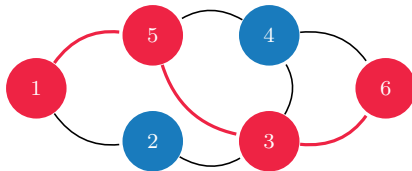


Movement in a graph

- **Def:** A **walk** of length l from v_0 to v_l is an alternating sequence

$$\{v_0, e_1, v_1, \dots, v_{l-1}, e_l, v_l\}, \text{ where } e_i \text{ is incident with } v_{i-1}, v_i$$

- A **trail** is a walk without repeated edges
- A **path** is a walk without repeated nodes (hence, also a trail)



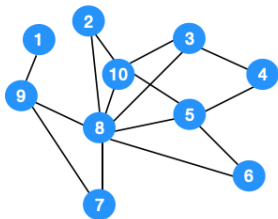
- A walk or trail is **closed** when $v_0 = v_l$.
- A **cycle** is a closed walk with no repeated nodes except $v_0 = v_l$

Image credit: G. Mateos

Graph matrices

Representing graphs with adjacency matrix

- Intuitive to implement
 - ⇒ Mathematical analysis / graph algorithm design
 - ⇒ Implementation up to hundred of thousands of nodes

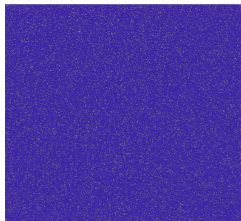


$$A = \begin{matrix} & \begin{matrix} \text{Node 1} & & & & & & & & & \text{Node 10} \end{matrix} \\ \begin{matrix} \text{Node 1} \\ \text{Node 2} \\ \text{Node 3} \\ \text{Node 4} \\ \text{Node 5} \\ \text{Node 6} \\ \text{Node 7} \\ \text{Node 8} \\ \text{Node 9} \\ \text{Node 10} \end{matrix} & \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \end{matrix}$$

- Matrices (arrays) are basic data objects in software environments
 - ⇒ Naive memory requirement is $O(N^2)$
 - ⇒ May be undesirable for large, sparse graphs

Sparsity of adjacency matrix

- ▶ Graph matrices are **super sparse**
⇒ 1000 nodes - 1M entries - **4.8% non-zero**



- ▶ Most real-world networks are sparse: $M \ll \frac{N(N-1)}{2}$
- ▶ Graph density: $\rho = \frac{M}{N^2}$

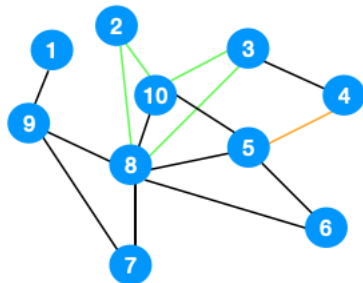
Network dataset	Order N	Graph density ρ
WWW (Stanford-Berkeley)	319,717	$\sim 10^{-4}$
Social network (LinkedIn)	6,946,668	$\sim 10^{-6}$
Communication (MSN IM)	242,720,596	$\sim 10^{-7}$
Collaboration (DBLP)	317,080	$\sim 10^{-4}$
Roads (California)	1,957,027	$\sim 10^{-6}$
Proteins (S. Cerevisiae)	1,870	$\sim 10^{-2}$

Table credit: J. Leskovec

Powers of adjacency matrix

- Powers of the adjacency matrix indicate a walk between two nodes

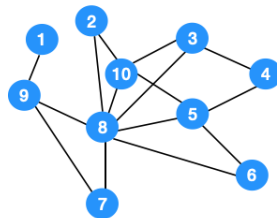
$$\mathbf{A}^2 = \begin{matrix} & \begin{matrix} \text{Node 3} \\ \text{Node 4} \end{matrix} & \\ \begin{matrix} \text{Node 2} \\ \text{Node 5} \end{matrix} & \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 2 & 2 & 0 & 2 & 1 & 1 & 1 & 1 & 1 \\ 0 & 2 & 3 & 0 & 3 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 2 & 0 & 1 & 0 & 2 & 0 & 2 \\ 0 & 2 & 3 & 0 & 4 & 1 & 1 & 2 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 2 & 1 & 1 & 1 & 2 \\ 1 & 1 & 1 & 0 & 1 & 1 & 2 & 1 & 1 & 1 \\ 1 & 1 & 1 & 2 & 2 & 1 & 1 & 7 & 1 & 3 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 3 & 1 \\ 0 & 1 & 1 & 2 & 1 & 2 & 1 & 3 & 1 & 4 \end{pmatrix} \end{matrix}$$



- Figure example:
 - ⇒ two walks of hop 2 connecting nodes 2 and 3
 - ⇒ zero walks of hop 2 connecting nodes 4 and 5

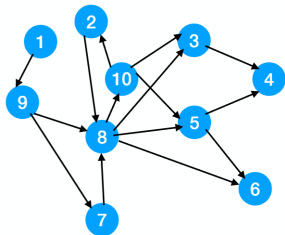
Representing graphs with lists

- ▶ An **adjacency list** representation is an array of size N
 - ⇒ **ith element** is a list of vertices adjacent to node i
 - ⇒ **Ex.** $L_a[1] = \{9\}$, $L_a[5] = \{4, 6, 8, 10\}$
- ▶ An **edge list** stores the node pairs incident to each edge
 - ⇒ **ith element** is a list of vertices adjacent to node i
 - ⇒ **Ex.** $L_e[1] = \{1, 9\}$, $L_e[2] = \{2, 8\}$, $L_e[3] = \{2, 10\} \dots$
- ▶ **Adjacency list**: ideal for large-scale implementation
 - ⇒ Exploits sparsity and fits in memory billion node graphs
 - ⇒ Memory requirement $\mathcal{O}(E)$
 - ⇒ Finds neighbors faster - **Google PageRank**



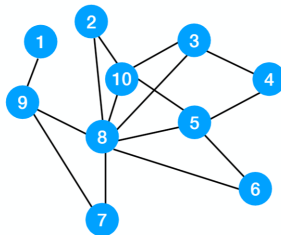
Directed vs. undirected graphs

Directed



$$\mathbf{A} = \begin{pmatrix} 0 & 0. & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & -1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & 1 & -1 & 0 & -1 & -1 & 1 & 0 & 1 & -1 \\ 1 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 0 & 0 \\ 0 & -1 & -1 & 0 & -1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Undirected

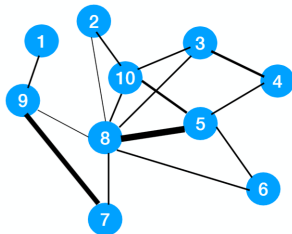


$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

27.png

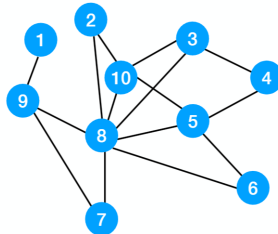
Weighted vs. unweighted graphs

Weighted



$$\mathbf{A} = \begin{pmatrix} 0 & 0.7 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.7 & 0 & 1 \\ 0 & 0 & 0 & 2 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 2 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 3 & 0 & 2 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.6 & 3 & 0 \\ 0 & 0.7 & 1 & 0 & 3 & 1 & 0.6 & 0 & 0.8 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 3 & 0.8 & 0 & 0 \\ 0 & 1 & 1 & 0 & 2 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Unweighted



$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Connected vs. disconnected graphs

Connected



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

Disconnected

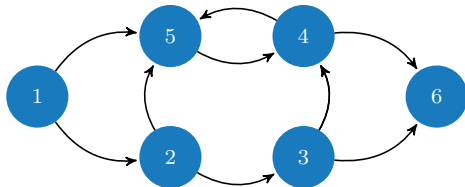


$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

- **Component:** a maximally connected subgraph
⇒ maximal means that adding a vertex ruins the connectivity
- **Ex.** Components are $\{1,2,3\}$, $\{4,5,6,7\}$
⇒ $\{5,6,7\}$ not maximal
- Disconnected graphs have 2 or more components
⇒ Largest component is often called **giant component**

Connectivity of directed graphs

- Connectivity is more subtle with directed graphs. Two notions
 - ⇒ **Strongly connected**: if for every pair $u, v \in \mathcal{V}$, u is reachable from v (via a directed walk) and vice versa
 - ⇒ **Weakly connected**: if connected after disregarding arc directions, i.e., the underlying undirected graph is connected

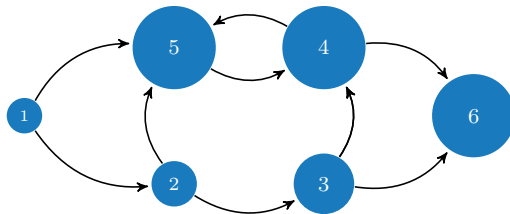


- Above graph is weakly connected but not strongly connected
 - ⇒ Strong connectivity obviously implies weak connectivity

Slide credit: G. Mateos

How well-connected nodes are?

- **Q:** Which node is the most connected?
- **A:** **Node rankings** to measure website relevance, social influence
- There are two important **connectivity indicators**
 - ⇒ How many links point to a node (outgoing links irrelevant)
 - ⇒ How important are the links that point to a node



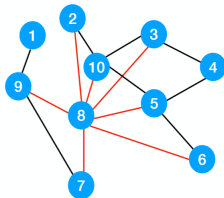
- Idea exploited by Google's PageRank to rank webpages
 - ⇒ ... by social scientists to study trust & reputation in social networks

Slide credit: G. Mateos

Degree matrix of undirected graphs

- ▶ The **degree** of node i is its number of incident edges
⇒ High-degree vertices likely influential, central, prominent

Undirected graph



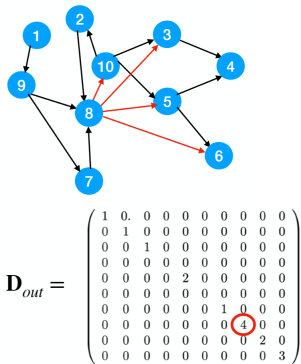
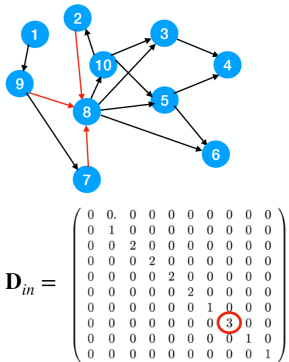
$$\mathbf{D} = \text{diag}(\mathbf{A}\mathbf{1})$$

$$\mathbf{D} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 7 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \end{pmatrix}$$

- ▶ **Degree matrix:** diagonal matrix with D_{ii} denoting the degree of i
⇒ **Ex.** $D_{88} = 7$ implies node v_8 has eight undirected edges
- ▶ The sum of the degrees is twice the size of the graph: $\sum_{i=1}^N D_{ii} = 2N$

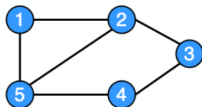
Degree matrix of directed graphs

- **In degree** the number of directed edges **incident** at node i
- **Out degree** the number of directed edges **leaving** node i



- For digraphs we also encode the direction of the arc, namely

$$B_{ij} = \begin{cases} 1, & \text{if edge } j \text{ is } (k, i) \\ -1, & \text{if edge } j \text{ is } (i, k) \\ 0, & \text{otherwise} \end{cases}.$$



$$\mathbf{B}_{un} = \begin{matrix} & \begin{matrix} (1,2) & (2,3) & (3,4) & (4,5) & (2,5) & (1,5) \end{matrix} \\ \begin{pmatrix} -1 & 0 & 0 & 0 & 0 & -1 \\ 1 & -1 & 0 & 0 & -1 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix} & \begin{matrix} \text{Node 1} \\ \text{Node 2} \end{matrix} \end{matrix}$$

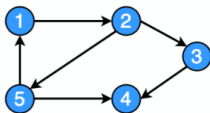
- Sign assigned on node label
 - ⇒ (1,2): node 1 (-) node 2(+)
 - ⇒ smallest label - negative sign

Incidence matrix

Directed graph

- ▶ A graph can be also represented by its $N \times M$ **incidence matrix** $\mathbf{B}$
 - $\Rightarrow \mathbf{B}$ is in general not a square matrix, unless $N = M$
- ▶ For undirected graphs, the entries of $\mathbf{B}$ are

$$B_{ij} = \begin{cases} 1, & \text{if vertex } i \text{ incident to edge } j \\ 0, & \text{otherwise} \end{cases}.$$



$$\mathbf{B}_{dir} = \begin{pmatrix} \text{edge} & (1,2) & (2,3) & (3,4) & (5,4) & (2,5) & (5,1) \\ \text{Node 1} & -1 & 0 & 0 & 0 & 0 & 1 \\ & 1 & -1 & 0 & 0 & -1 & 0 \\ & 0 & 1 & -1 & 0 & 0 & 0 \\ & 0 & 0 & 1 & 1 & 0 & 0 \\ \text{Node 5} & 0 & 0 & 0 & -1 & 1 & -1 \end{pmatrix}$$

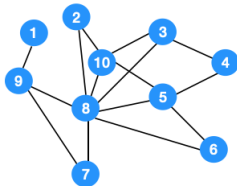
- ▶ Positive sign: edge incident on node
- ▶ Negative sign: edge leaving from node

Graph Laplacian

For undirected graphs

- Laplacian matrix: $\mathbf{L} = \mathbf{D} - \mathbf{A} = \mathbf{B}\mathbf{B}^\top \in \mathbb{R}^{N \times N}$

⇒ Degrees on the main diagonal



$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & 3 & -1 & 0 & 0 & 0 & -1 & 0 & -1 \\ 0 & 0 & -1 & 2 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 4 & -1 & 0 & -1 & 0 & -1 \\ 0 & 0 & 0 & 0 & -1 & 2 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 2 & -1 & -1 & 0 \\ 0 & -1 & -1 & 0 & -1 & -1 & -1 & 7 & -1 & -1 \\ -1 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & 3 & 0 \\ 0 & -1 & -1 & 0 & -1 & 0 & 0 & -1 & 0 & 4 \end{pmatrix}$$

- Diagonally dominant matrix: each row sums to zero
- Positive semidefinite matrix: $\mathbf{x}^\top \mathbf{L} \mathbf{x} \geq 0$ for any vector $\mathbf{x} \neq \mathbf{0}$

Graph shift operator

- ▶ A graph shift operator (GSO) matrix $\mathbf{S}$ is **any $N \times N$ matrix** that captures the structure of the graph
 - $\Rightarrow$ Adjacency matrix $\mathbf{A}$ and Laplacian matrix $\mathbf{L}$ are graph shift operator (GSO) matrices $\mathbf{S}$
 - $\Rightarrow$ **Formal definition:** $S_{uv} \neq 0 \Rightarrow u = v$, or $(u, v) \in \mathcal{E}$
- ▶ Other candidates for $\mathbf{S}$ are the normalized matrices
 - $\Rightarrow$ **Normalized adjacency:** $\mathbf{A}_n = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$
 - $\Rightarrow$ **Normalized Laplacian:** $\mathbf{L}_n = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$
 - $\Rightarrow$ **Random-walk Laplacian:** $\mathbf{L}_{rw} = \mathbf{D}^{-1} \mathbf{L}$

Features

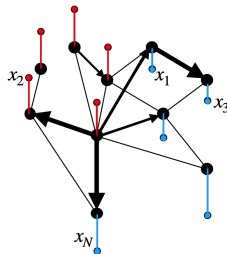
Node features

- Scalar values over each node:

- ⇒ collect in vector $\mathbf{x} = [x_1, x_2, \dots, x_N]^T \in \mathbb{R}^N$

- ⇒ scalar node feature

- ⇒ also known as a graph signal



- Graph signals can represent different quantities

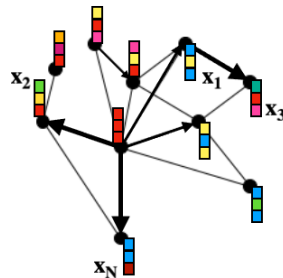
- ⇒ **Measurements of physical phenomenon:** temperature observed at weather stations over a temperature monitoring network

- ⇒ **Node labels:** user opinion on a social network with users as nodes

- ⇒ **Node class probabilities:** Probabilities of nodes belonging to a certain group in a community clustering problem

Multi-dimensional Node features

- Vector of values over each node
 - ⇒ Collect all in a matrix $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{F \times N}$
 - ⇒ Ex. gender, hair color, country of origin in social networks
 - ⇒ Ex. temperature, humidity, wind speed in sensor networks

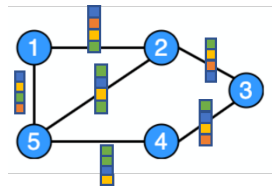


- Can be decomposed into F scalar graph signals
 - ⇒ Can process each individually and then aggregate (more later on)
 - ⇒ Often we will explain concepts with scalar values on nodes for simplicity

Edge features

- Each edge is associated with a **scalar** (e.g., edge weight) or a collection of feature values
 - ⇒ **Ex.** water flow over pipes in a water distribution network; type of pipe, diameter, material etc.
 - ⇒ **Ex.** traffic flow in a road transportation network
 - ⇒ Collect them into an edge feature matrix

$$\mathbf{F} = \begin{bmatrix} f_{11} & \dots & f_{1F} \\ f_{21} & \dots & f_{2F} \\ \dots & \dots & \dots \\ f_{E1} & \dots & f_{EF} \end{bmatrix} \in \mathbb{R}^{|\mathcal{E}| \times F}$$

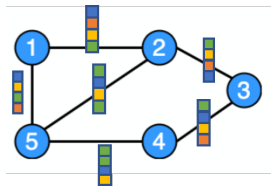


Edge features

- Each edge is associated with a **scalar** (e.g., edge weight) or a collection of feature values
 - ⇒ **Ex.** water flow over pipes in a water distribution network; type of pipe, diameter, material etc.
 - ⇒ **Ex.** traffic flow in a road transportation network
 - ⇒ Collect them into an edge feature matrix

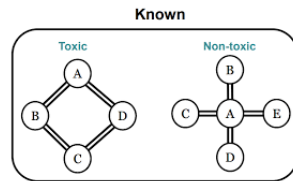
$$\mathbf{F} = \begin{bmatrix} f_{11} & \dots & f_{1F} \\ f_{21} & \dots & f_{2F} \\ \dots & \dots & \dots \\ f_{E1} & \dots & f_{EF} \end{bmatrix} \in \mathbb{R}^{|\mathcal{E}| \times F}$$

- Handling edge features
 - ⇒ **Split** them as different graphs where each feature indicates a type of connection - work with **multiple graphs** in parallel
 - ⇒ **Transform** them into a single function $f((f_{e1}, \dots, f_{eF}), \Theta) \rightarrow a_e = a_{ij}$ -weight for edge $e = (i, j)$
 - ⇒ Use them as they are and input to a ML system



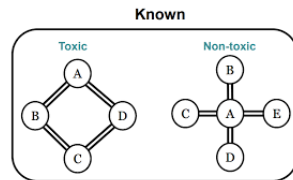
Graph-level features

- **Graph-level features:** characteristics or properties of the whole graph
 - ⇒ **Ex.** Is the molecule toxic or non-toxic?
 - ⇒ **Ex.** Is the water network resilient to flooding?
 - ⇒ **Ex.** More abstract features are also possible; average path length, degree distribution, graph density, clustering coefficient, etc.

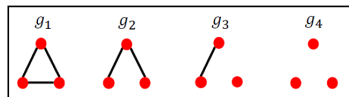


Graph-level features

- **Graph-level features:** characteristics or properties of the whole graph
 - ⇒ **Ex.** Is the molecule toxic or non-toxic?
 - ⇒ **Ex.** Is the water network resilient to flooding?
 - ⇒ **Ex.** More abstract features are also possible; average path length, degree distribution, graph density, clustering coefficient, etc.



- Bag of Graphlets (BoG) is a method of capturing the **structural information** of a graph
 - ⇒ Graphlets are **small, connected subgraphs** that capture local structural patterns within a graph
 - ⇒ Each graphlet in the Bag of Graphlets represents a **unique local structural pattern** within the graph
 - ⇒ By counting the occurrences of different graphlets, we create a feature vector representing the graph's structure



How to build graphs?

How to build graphs?

- ▶ We have two main strategies to build graphs
- ▶ **Physical networks:** the edges are already available
 - ⇒ **Road network:** nodes are cross roads, edges are streets connecting them
 - ⇒ **Robot networks:** nodes are agents, edges are communication links
 - ⇒ **Brain networks:** nodes are brain areas, edges are structural connectivities

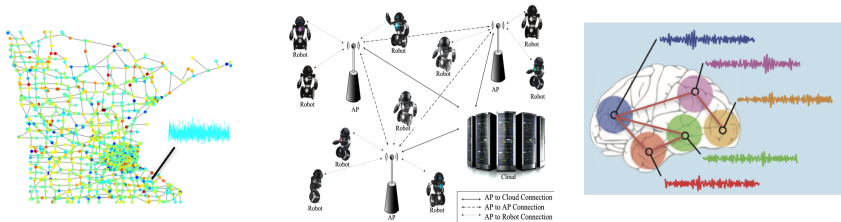


image credit Varma et al

- ▶ **Abstract networks:** build a **proximity structure** from data points
 - ⇒ These can be dependencies, similarities, causal influence etc.

Abstract networks

- ▶ Depend on proximities between data points
 - ⇒ Let $\mathbf{x}_1, \dots, \mathbf{x}_N$ be N datapoints each of dimensions F ; i.e., $\mathbf{x}_i \in \mathbb{R}^F$
 - ⇒ **Treat** each datapoint as **a node**
 - ⇒ **Connect** two datapoints i and j based on a **proximity/similarity/distance metric**, e.g.,:

$$sim(i, j) = g(\mathbf{x}_i, \mathbf{x}_j)$$

- ⇒ Leads to a **fully-connected** graph
- ▶ Various choices imply various structures
 - ⇒ **Similarity**: correlation; dependency; causation; covariance;
 - ⇒ **Sparsity**: threshold; k - nearest neighbors; ϵ -ball neighborhood
 - ⇒ **Hyperparameter**: k, ϵ

- **Pearson correlation:** correlated datapoints provide information to each-other

⇒

$$\text{sim}(i, j) = \rho_{ij} = \frac{\text{cov}(\mathbf{x}_i, \mathbf{x}_j)}{\sqrt{\text{var}[\mathbf{x}_i]\text{var}[\mathbf{x}_j]}}, \quad i, j \in \mathcal{V}$$

⇒ Build edge set as $\mathcal{E} = \{(i, j) : \rho_{ij} \neq 0\}$ or $\mathcal{E} = \{(i, j) : \rho_{ij} > \gamma\}$

⇒ Parameter γ defines the sparsity

⇒ Typically sparser graphs are preferred: **reduce** spurious correlation effects and estimation error

- **Other choices:** conditional correlation, partial correlation, cosine similarity⁽¹⁾

ϵ – neighbourhood networks

- Let $\text{dist}(\mathbf{x}_i, \mathbf{x}_j)$ represent a distance between datapoints (e.g., physical or Euclidean distance)
 - ⇒ Set edge weights as the **Kernelized** version of the distance; e.g., Gaussian kernel

$$A_{ij} = \begin{cases} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma^2}\right) = \exp\left(-\frac{\text{dist}^2(\mathbf{x}_i, \mathbf{x}_j)}{2\sigma^2}\right) & \text{if } \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \leq \epsilon \\ 0 & \text{otherwise} \end{cases}$$

- ⇒ Build edge set as $\mathcal{E} = \{(i, j) : \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \leq \epsilon\}$
- ⇒ σ is a hyperparameter of the kernel; ϵ defines the radius
- ⇒ $\epsilon \rightarrow 0$ sparser graphs; $\epsilon \rightarrow \infty$ densely connected graphs

- **Q:** How does σ influence connectivity for a fixed ϵ ?

k – nearest neighbourhood (k NN) networks

- Build the **similarities/distances** between datapoints $\text{sim}(i, j)$
 - ⇒ Connect each node to its k – most **similar/closest** neighbors
 - ⇒ Leads to a directed graph: often directionality is removed
 - ⇒ Higher k a more densely connected graph

k – nearest neighbourhood (k NN) networks

- Build the **similarities/distances** between datapoints $\text{sim}(i, j)$
 - ⇒ Connect each node to its k – most **similar/closest** neighbors
 - ⇒ Leads to a directed graph: often directionality is removed
 - ⇒ Higher k a more densely connected graph

Questions?

Further reading

- ▶ (Barabasi) Network science <http://networksciencebook.com>
⇒ Ch1 and Ch2
 - ▶ (Isufi et al) Graph filters for processing and learning from network data
⇒ Ch1 (excluding Sec. 1.2)
 - ▶ (Ma and Tang) Deep Learning on Graphs https://web.njit.edu/~ym329/dlg_book/
⇒ Ch 2 (excluding Sec. 2.3.3; 2.4; 2.5)
-
- ▶ Some of the material in these slides (figures, tables etc.) were taken from these courses:
⇒ CS2242: <https://web.stanford.edu/class/cs224w/>
⇒ ICASSP 2022 short-course: <https://iie.fing.edu.uy/personal/flarroca/teaching/signal-processing-and-learning-from-network-data/>

Review: mathematical notation and concepts

- ▶ Throughout the course, we will denote mathematical variable as
 - ⇒ plain letters e.g., a –scalar variables
 - ⇒ bold lowercase, e.g., $\mathbf{a} = [a_1, \dots, a_N]^\top \in \mathbb{R}$ –a column vector containing N real entries
 - ⇒ bold uppercase, e.g., $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_M] \in \mathbb{R}^{N \times M}$ –a matrix with real entries of dimensions $M \times N$
 - ⇒ caligraphic letters, e.g., $\mathcal{S}$ –set variables
 - ⇒ the transpose of matrix $\mathbf{A}$ is $\mathbf{A}^\top$
 - ⇒ the hermitian of matrix $\mathbf{A}$ is $\mathbf{A}^H$
- ▶ *Diagonal dominant matrix*: $\mathbf{A}$ with $|A_{ii}| \geq \sum_{j \neq i} |A_{ij}|$, $n = 1 : N$
- ▶ *Positive semidefinite*: $\mathbf{A}$ with $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$, for all $\mathbf{x} \neq \mathbf{0}_N$
- ▶ *Positive definite*: $\mathbf{A}$ with $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$, for all $\mathbf{x} \neq \mathbf{0}_N$
- ▶ (Isufi et al) Graph filters for processing and learning from network data
 - ⇒ Appendix A